

PAPER

Enhanced Side-Channel Cube Attacks on PRESENT

Xinjie ZHAO^{†a)}, Shize GUO^{††}, Fan ZHANG^{†††}, Tao WANG[†], Zhijie SHI^{†††}, Nonmembers,
and Hao LUO^{††††}, Member

SUMMARY This paper proposes several improved Side-channel cube attacks (SCCAs) on PRESENT-80/128 under single bit leakage model. Assuming the leakage is in the output of round 3 as in previous work, we discover new results of SCCA on PRESENT. Then an enhanced SCCA is proposed to extract key related non-linear equations. 64-bit key for both PRESENT-80 and 128 can be obtained. To mount more effective attack, we utilize the leakage in round 4 and enhance SCCA in two ways. A partitioning scheme is proposed to handle huge polynomials, and an iterative scheme is proposed to extract more key bits. With these enhanced techniques, the master key search space can be reduced to 2^8 for PRESENT-80 and to 2^{29} for PRESENT-128.

key words: Side-channel cube attack, PRESENT, Single bit leakage

1. Introduction

Side-Channel Cube Attack (SCCA) [8] is a powerful attack that combines Side-channel attack (SCA) [13]–[15] and cube attack [6], [7]. The cube attack [6] can be launched on any cryptosystem where an intermediate bit can be represented by a low degree multivariate polynomial of public (e.g., plaintext) and secret (e.g., key) variables. An interesting feature of the cube attack is that it only requires a black-box access to a target cryptosystem and may be applied even if only a few output bits can be accessed by an adversary. To date, most of the cube attacks are applied to ciphers with reduced rounds [3], [7].

When combined with SCAs that exploit the leakages from the cipher implementations, cube attacks turn into SCCAs that can attack full-round ciphers in practice. Different from SCA, which is normally mounted to the first or the last round, SCCA can explore the leakages of deeper rounds, as Algebraic side-channel attacks [16], [20] did. Meanwhile, SCCA can utilize any type of leakages and is a generic tool for SCA. Moreover, inherited with the advantages of cube attacks, SCCA might be launched under the black-box setting where the full design of the cipher is unknown [3], [8]. Due to its realistic and serious threats, further investigation

on SCCA is necessary to evaluate the security of block ciphers.

In SCCAs, leakage model determines where the information is leaked and how much information can be obtained by adversaries. One of the well known models is *Single bit leakage model* (SBLM). In this model, the single bit value of the intermediate state can be obtained from probing [12]. Compared with other leakage models like *Hamming weight leakage model* (HWLM), SBLM is more costly because it requires measurements with high accuracy. However, SBLM also has some advantages. The number of chosen plaintexts required in SBLM is small. In addition, the scale of the polynomial generated from SBLM is low, resulting that it is possible to utilize the leakage in deeper rounds. The extraction of key related equations is also fast.

SCCA has been applied to evaluate the physical security of many ciphers, especially to those having simple iterative structures, such as PRESENT [2], [18], NOEKEON [1] KATAN [4] and Hummingbird-2 [9]. In this paper, we investigate new techniques to enhance SCCA and apply them to PRESENT. PRESENT has two versions with different key lengths: PRESENT-80 and PRESENT-128. This paper considers SCCA on both versions.

Related work

Table 1 summarizes the previous SCCA results on PRESENT and compares them with the work in this paper.

In 2009, Yang et al. proposed the first SCCA on PRESENT-80 [18]. The attack is based on SBLM in round 3 by extracting linear equations on the secret key bits. They use 2^{15} chosen plaintexts to recover 48 key bits, as listed in row 1 of Table 1. In 2011, Abdul-Latip et al. [2] proposed an extended SCCA on PRESENT based on HWLM. Assuming that 64-bit Hamming weight of the first round output is available, they spent several weeks to extract key related linear and non-linear (quadratic) equations. Using about 2^{13} chosen plaintexts, they can recover the 64-bit first round key

Manuscript received April 9, 2012.

Manuscript revised August 13, 2012.

[†]The authors are with the Department of Computer Engineering, Ordnance Engineering College, Shijiazhuang 050003, China.

^{††}The author is with the Institute of North Electronic Equipment, Beijing 100083, China.

^{†††}The author is with the Department of Computer Science and Engineering, University of Connecticut, Storrs 06269, USA.

^{††††}The author is with the School of Aeronautics and Astronautics, Zhejiang University, Hangzhou 310027, China.

a) E-mail: zhaoxinjieem@163.com

DOI: 10.1587/transfun.E96.A.332

Table 1 Comparisons of this paper with previous work.

PRESENT	Ref.	Leakage model	Samples	key
PRESENT-80	[18]	SBLM in round 3	2^{15}	2^{32}
PRESENT-80	[2]	HWLM in round 1	2^{13}	2^{16}
PRESENT-80	this paper	SBLM in round 3	$2^{10.89}$	2^8
PRESENT-128	[2]	HWLM in round 1	2^{13}	2^{64}
PRESENT-128	this paper	SBLM in round 3	$2^{10.88}$	2^{54}
PRESENT-128	this paper	SBLM in round 4	$2^{14.57}$	2^{29}

for PRESENT-80/128, as shown in rows 2 and 4 in Table 1.

Contributions

Focusing on SBLM, this paper proposes several techniques to enhance SCCA on PRESENT and achieves better results than previous work. The main contributions are summarized below.

(1) New results of SCCAs on PRESENT (Sect. 3). With a leakage bit in round 3, we show that, more cubes of smaller size can be extracted to recover the secret key and the number of the chosen plaintexts for the attack can be reduced in practice. To reveal 48 key bits of PRESENT, only $2^{10.88}$ chosen plaintexts are required, instead of 2^{15} in [18]. We also observe that, in practice, many chosen plaintexts for different cubes are duplicated and all previous SCCA results, e.g., in [2], [18], are inaccurate.

(2) An enhanced SCCA with non-linear equations (Sect. 4). To improve SCCA, we extract the non-linear equations on the secret key bits. Then, the 64-bit first round key of both PRESENT-80 and PRESENT-128 can be extracted without increasing the number of chosen plaintexts.

(3) An enhanced SCCA with partitioning (Sect. 5). To launch more effective SCCA on PRESENT-128, we move the leakage bit to round 4. However, the polynomial becomes too large to be represented in the conventional SCCA. For this, we propose a partitioning scheme to handle large polynomials. The proposed scheme describes the strategy of how to cut the large polynomials into several pieces while the related cubes can still be extracted independently. 60 key bits of PRESENT-128 with $2^{14.56}$ chosen plaintexts can be obtained with the partitioning scheme.

(4) Iterative SCCAs (Sect. 6). To recover more key bits, we propose an iterative SCCA technique to reduce the complexity of the original polynomial. 72 key bits of PRESENT-80 and 99 key bits of PRESENT-128 can be extracted with $2^{10.89}$ and $2^{14.57}$ chosen plaintexts, respectively. Both are the best results of SCCA on PRESENT.

2. Preliminaries

2.1 PRESENT Block Cipher

PRESENT is a 31-round block cipher with SPN structure. The block size is of 64 bits. In the encryption procedure, the input of round 1 is the plaintext XOR-ed with subkey K^1 , and the ciphertext is the output of round 31 XOR-ed with subkey K^{32} . Each round consists of three major operations.

1. addRoundKey-AK. At the beginning of each round, 64-bit output of the previous round function is XOR-ed with the subkey.
2. sBoxlayer-SL. Sixteen identical S-Boxes are used in parallel. Each S-Box maps a 4-bit input (x_0, x_1, x_2, x_3) to a 4-bit output (y_0, y_1, y_2, y_3) .
3. pLayer-PL. The i -th bit is moved to position $P(i)$ by a permutation table P .

Below is the key schedule of the PRESENT-80 (please

refer to [5] for the key schedule of PRESENT-128). The 80-bit key is stored in a register $K = k_{79}k_{78} \dots k_0$. At Round i ($1 \leq i \leq 31$), the 64-bit round key K^i is the 64 leftmost bits of K . After K^i is extracted, K is rotated by 61 bits to the left, then a S-Box is applied to the left-most 4 bits of K . Finally, the round-counter i is XOR-ed with bits $k_{19}k_{18}k_{17}k_{16}k_{15}$.

2.2 Cube Attack and Side-Channel Cube Attack

The cube attacks are described briefly here. More detailed descriptions can be referred to [3], [6], [7]. The attack can be divided into two phases. A preprocessing phase determines which queries should be sent to a black-box oracle. An online phase deduces key related low-degree equations by querying the oracle with chosen public variables. Finally the key can be recovered by solving those equations.

Suppose a cipher has m public variables $V = \{v_1, \dots, v_m\}$ and n key variables $K = \{k_1, \dots, k_n\}$. Let $X = V \cup K$. Any intermediate bit in the cipher can be described by a multivariate polynomial $f(X)$. Let the degree of $f(X)$ be $D_{eg}(f)$.

In the preprocessing phase, the adversary randomly chooses a subset of public variables from V . Let I be the set of the index of the selected public variables. Apparently, $I \subset \{1, \dots, m\}$. I is called a *cube*. An element in I is called a *cube index*. Let t_I be the product of the selected variables. t_I is called a *maxterm*. $f(X)$ can be represented as

$$f(X) = f(v_1, \dots, v_m, k_1, \dots, k_n) = t_I \bullet p_{S(I)} + q_I(X) \quad (1)$$

$p_{S(I)}$ is called a *superpoly* of I . It is also a maxterm related equation. q_I contains all monomials that are not divisible by t_I . For example, below is a polynomial $f(X)$ of degree 4 with 8 variables and 17 monomials:

$$\begin{aligned} f(X) = & v_1v_2v_3k_1 + v_1v_2v_3k_2 + v_1v_4k_1 + v_1v_4k_3 \\ & + v_2v_4k_4 + v_3v_4k_2 + v_1v_2v_3 + v_1k_1k_2 + v_1k_2k_3 \\ & + v_1k_1 + v_2v_4 + v_3v_4 + k_2k_3 + k_2k_4 + v_3 + v_4 + 1 \end{aligned} \quad (2)$$

$f(X)$ can be rewritten by merging monomials with the same public variables:

$$\begin{aligned} f(X) = & v_1v_2v_3(k_1 + k_2 + 1) + v_1v_4(k_1 + k_3) \\ & + v_2v_4(k_4 + 1) + v_3v_4(k_2 + 1) + v_1(k_1 + k_1k_2 + k_2k_3) \\ & + v_3 + v_4 + k_2k_3 + k_2k_4 + 1 \end{aligned} \quad (3)$$

Let $I = \{1, 2, 3\}$ be a cube of size 3 and 1, 2, 3 are cube indexes. Then $t_I = v_1v_2v_3$, $p_{S(I)} = k_1 + k_2 + 1$, $q_I = v_1v_4(k_1 + k_3) + \dots + 1$. Given an assignment to the public variables, f can be reduced to a function that has only key variables. If all public variables in t_I (called *tweakable cube variables*) can take either 0 or 1 while other public variables are fixed to 0, the symbolic sum over $GF(2)$ of all the function deduced from $f(X)$ is exactly $p_{S(I)}$. In the example, if $v_4=0$, there are eight assignments to (v_1, v_2, v_3, v_4) for $f(X)$ in Eq. (3). For each assignment, the value of f can be evaluated from

Table 2 Notations used in SCCA on PRESENT.

Variable	Notations
$N_{S(I)}$	cube size of I
$S_{S(I)}$	summation of indexes in I
k_i	the i -th bit of master key
K^i	subkey in Round i
$p_{S(I)}$	superpoly of I
$+$	exclusive or (XOR)
L_i^j	leakage bit, the j -th bit of S-Box in Round i
$f(L_i^j)$	polynomial representation of L_i^j
$N_m(L_i^j)$	number of monomials in $f(L_i^j)$
$Deg(L_i^j)$	polynomial degree of $f(L_i^j)$
N_{cp}	number of chosen plaintexts
N_{rk}	number of recovered key bits

either the oracle or side-channels. The sum of f 's value on the eight assignments is the value of the superpoly $k_1 + k_2 + 1$.

The previous process could be repeated to get more low-degree superpolys. For instance, if we only extract the linear superpolys, $t_1 = v_1v_4$ when $p_{S(I)} = k_1 + k_3$; $t_1 = v_2v_4$ when $p_{S(I)} = k_4 + 1$; $t_1 = v_3v_4$ when $p_{S(I)} = k_2 + 1$ and $t_1 = v_1v_2v_3$ when $p_{S(I)} = k_1 + k_2 + 1$. If we only utilize four linear superpolys, (k_1, k_2, k_3, k_4) can be obtained with $3 \cdot 2^2 + 2^3 = 20$ chosen plaintexts. For non-linear superpolys, $t_1 = v_1$, $p_{S(I)} = k_1 + k_1k_2 + k_2k_3$ when $I=\{1\}$.

Side-Channel Attack (SCA) deduces the value of f for the leakage bit depending on the measurement capability. Cube attack can then use the deduced values of f to solve the equation systems composed of superpolys and retrieve key bits. With the aid of SCAs, the cube attack becomes practical. The new type of attack that combines the SCA and cube attacks is called as Side-Channel Cube Attack (SCCA). As to the recent proposed lightweight block ciphers, such as PRESENT, which use S-Boxes of small dimensions (thus resulting in lower degree equations), SCCA is quite effective. This paper mainly focuses on SBLM based SCCA in the deeper rounds, e.g, round 3 or round 4 of PRESENT.

2.3 Notations

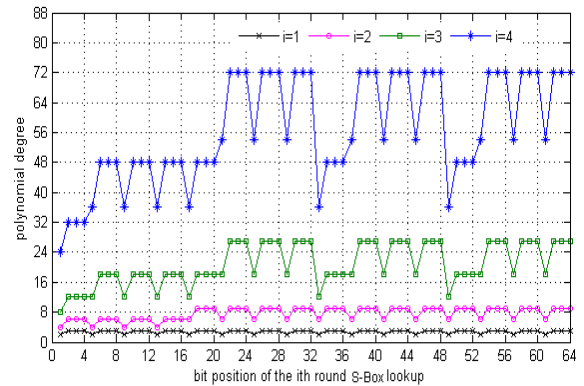
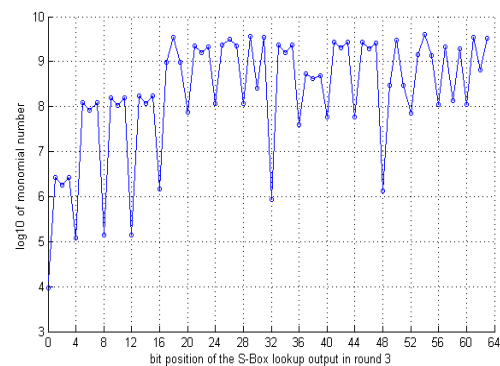
Table 2 lists some notations used in this paper.

3. Implemented SCCA on PRESENT-80 with New Results

3.1 Locate the Target Leakage Bit

In SCCA, the choice of the leakage bit L_i^j is based on three criterions to reduce the scale of f . (1) f should be represented by as many key variables as possible, maximizing the number of key bits to be extracted. (2) Deg is low. And (3) N_m is small. Usually, the first criterion is used to locate the best leakage round, the last two are used to locate the best bit in the round.

Round 3 is the first place where all the 64 bit of K^1 are involved in $f(L_i^j)$. So we choose $i \geq 3$. Figure 1 shows the value of $Deg(L_i^j)$ where $1 \leq i \leq 4$ and $0 \leq j \leq 63$. We can see that the degree of L_3^j is much lower than L_4^j . Next we

**Fig. 1** $Deg(L_i^j)$ in rounds $i = 1 \dots 4$.**Fig. 2** $N_m(L_3^j)$ in round 3.

consider which bit of L_3^j we should choose. Figure 2 shows how $N_m(L_3^j)$ changes with j . The numbers of monomials for $L_3^0, L_3^4, L_3^8, L_3^{12}$ are relatively small. So SCCA on L_3^0, L_3^4, L_3^8 , or L_3^{12} has lower complexities.

3.2 Implemented Attack and New Results

We write the analysis code of SCCA running under an AMD Athlon 64 Dual core 3600+ processor cadenced at 2.0GHz. To speed up the process and only extract the linear superpolys, we adopt the same strategy as in [18]. For each cube, we only merge the monomials having at most one key variable or the monomials only involving public variables. Taking Eq. (2) as an example, we discard $v_1k_1k_2, v_1k_2k_3, k_2k_3, k_2k_4$ and get $f(X) = v_1v_2v_3(k_1 + k_2 + 1) + v_1v_4(k_1 + k_3) + v_2v_4(k_4 + 1) + v_3v_4(k_2 + 1) + v_1k_1 + v_3 + v_4 + 1$.

In our experiments, as to SCCA on L_3^0 , 32 key bits can be recovered with $2^{7.17}$ chosen plaintexts. As to SCCA on L_3^i ($i \in \{4, 8, 12\}$), 48 key bits can be extracted with $12 \times 2^2 + 24 \times 2^5 + 12 \times 2^8 \approx 2^{11.92}$ chosen plaintexts, which is much less than 2^{15} in [18]. According to [6], [7], in any d -random polynomial, any term t_i which is the product of $d-1$ public variables v_i has an extremely high probability to be a maxterm, and its corresponding superpoly is a polynomial of degree at most 1. [18] first considered this property and extracted many superpolys related with $d-1$ public variables. As $Deg(L_3^{12})=12$, so [18] got many superpolys related with

Table 3 Cube indexes and superpolys on L_3^{12} (PRESENT-80).

I	$PS(I)$	I	$PS(I)$
1, 2	$1 + k_{16}$	16, 17, 18, 33, 34	$1 + k_{48}$
0, 2	$k_{17} + k_{19}$	16, 17, 18, 32, 34	$k_{49} + k_{51}$
0, 1	$k_{18} + k_{19}$	16, 17, 18, 32, 33	$k_{50} + k_{51}$
13, 14	$1 + k_{28}$	16, 17, 18, 45, 46	$1 + k_{60}$
12, 14	$k_{29} + k_{31}$	16, 17, 18, 44, 46	$k_{61} + k_{63}$
12, 13	$k_{30} + k_{31}$	16, 17, 18, 44, 45	$k_{62} + k_{63}$
49, 50	$1 + k_{64}$	53, 54, 56, 57, 58	$1 + k_{68}$
48, 50	$k_{65} + k_{67}$	52, 54, 56, 57, 58	$k_{69} + k_{71}$
48, 49	$k_{66} + k_{67}$	52, 53, 56, 58, 59	$k_{70} + k_{71}$
61, 62	$1 + k_{76}$	52, 53, 54, 57, 58	$1 + k_{72}$
60, 62	$k_{77} + k_{79}$	52, 53, 54, 56, 58	$k_{73} + k_{75}$
60, 61	$k_{78} + k_{79}$	52, 53, 54, 56, 57	$k_{74} + k_{75}$
5, 6, 8, 9, 10	$1 + k_{20}$	21, 22, 24, 25, 26, 32, 33, 34	$1 + k_{36}$
4, 6, 8, 9, 10	$k_{21} + k_{23}$	20, 22, 24, 25, 26, 32, 33, 34	$k_{37} + k_{39}$
4, 5, 8, 9, 10	$k_{22} + k_{23}$	20, 21, 24, 25, 26, 32, 33, 34	$k_{38} + k_{39}$
4, 5, 6, 9, 10	$1 + k_{24}$	20, 21, 22, 25, 26, 32, 33, 34	$1 + k_{40}$
4, 5, 6, 8, 10	$k_{25} + k_{27}$	20, 21, 22, 24, 26, 32, 33, 34	$k_{41} + k_{43}$
4, 5, 6, 8, 9	$k_{26} + k_{27}$	20, 21, 22, 24, 25, 32, 33, 34	$k_{42} + k_{43}$
17, 18, 32, 33, 34	$1 + k_{32}$	16, 17, 18, 37, 38, 40, 41, 42	$1 + k_{52}$
16, 18, 32, 33, 34	$k_{33} + k_{35}$	16, 17, 18, 36, 38, 40, 41, 42	$k_{53} + k_{55}$
16, 17, 32, 33, 34	$k_{34} + k_{35}$	16, 17, 18, 36, 37, 40, 41, 42	$k_{54} + k_{55}$
29, 30, 32, 33, 34	$1 + k_{44}$	16, 17, 18, 36, 37, 38, 41, 42	$1 + k_{56}$
28, 30, 32, 33, 34	$k_{45} + k_{47}$	16, 17, 18, 36, 37, 38, 40, 42	$k_{57} + k_{59}$
28, 29, 32, 33, 34	$k_{46} + k_{47}$	16, 17, 18, 36, 37, 38, 40, 41	$k_{58} + k_{59}$

cube size 11. In fact, we observe that these superpolys can be extracted with smaller cube size, e.g., 5 or 8, and the chosen plaintext number can be further reduced. The cube indexes and superpolys of L_3^{12} are shown in Table 3.

3.3 New Observations on the Number of Chosen Plaintexts

In the attack, we observe that: when plaintexts are chosen based on the cubes identified in Table 3, many of them are actually duplicated. Considering all duplicated chosen plaintexts in Table 3, $N_{cp}(L_3^{12})$ can be further reduced to $2^{10.88}$. All existing SCCAs [1], [2], [4], [8], [18] simply counted the number of chosen plaintexts based on the number of cubes with their sizes.

Our observations can be used to reduce the chosen plaintexts needed in practice and estimate the data complexity of the attack more accurately. Below just lists the updated number of chosen plaintexts for some existing SCCAs on PRESENT. As to SBLM based SCCA in [18], the updated chosen plaintext number is $2^{12.50}$ instead of 2^{15} . As to HWLM based SCCA in [2], the updated number is $2^{12.27}$ instead of 2^{13} .

4. Enhanced SCCA with Non-linear Equations

As reported in [2], [6], [7], [19], utilizing low degree non-linear equations in addition to linear ones, can extract more key bits. In this section, we adopt the approach and extend SCCA with non-linear equations. Especially, we focus on the quadratic equations. Combined with the new observations in Sect. 3.3, we show that SCCA utilizing non-linear equations does not need to introduce new chosen plaintexts.

Table 4 Cube indexes and nonlinear superpolys on L_3^{12} (PRESENT-80).

I	$PS(I)$	I	$PS(I)$
2,	$k_{17} + k_{16}k_{17} + k_{16}k_{19}$	16, 17, 18, 34	$k_{49} + k_{48}k_{49} + k_{48}k_{51}$
14	$k_{29} + k_{28}k_{29} + k_{28}k_{31}$	16, 17, 18, 46	$k_{61} + k_{60}k_{61} + k_{60}k_{63}$
50	$k_{65} + k_{64}k_{65} + k_{64}k_{67}$	54, 56	$k_{69} + k_{68}k_{69} + k_{68}k_{71}$
62	$k_{77} + k_{76}k_{77} + k_{76}k_{79}$	55, 58	$k_{73} + k_{72}k_{73} + k_{72}k_{75}$
6, 8	$k_{21} + k_{20}k_{21} + k_{20}k_{23}$	22, 24, 25, 26, 32, 33, 34	$k_{37} + k_{36}k_{37} + k_{36}k_{39}$
4, 10	$k_{25} + k_{24}k_{25} + k_{24}k_{27}$	20, 21, 22, 26, 32, 33, 34	$k_{41} + k_{40}k_{41} + k_{40}k_{43}$
18, 32, 33, 34	$k_{33} + k_{32}k_{33} + k_{32}k_{35}$	16, 17, 18, 38, 40, 41, 42	$k_{53} + k_{52}k_{53} + k_{52}k_{55}$
30, 32, 33, 34	$k_{45} + k_{44}k_{45} + k_{44}k_{47}$	16, 17, 18, 36, 37, 38, 42	$k_{57} + k_{56}k_{57} + k_{56}k_{59}$

4.1 Extracting Non-linear Equations

The basic idea of extracting non-linear equations is that, for each cube, we only keep the monomials that have at most two key variables or the monomials only involving public variables; other monomials are discarded.

Taking Eq. (2) as an example, we can get $f(X) = v_1v_2v_3(k_1 + k_2 + 1) + v_1v_4(k_1 + k_3) + v_2v_4(k_4 + 1) + v_3v_4(k_2 + 1) + v_1(k_1 + k_1k_2 + k_2k_3) + v_3 + v_4 + 1$. One non-linear superpoly $p_{S(I)} = k_1 + k_1k_2 + k_2k_3$ for ($t_I = v_1$) can be extracted besides the linear equations. Selecting only four equations $k_1 + k_3, k_4 + 1, k_2 + 1$, and $k_1 + k_1k_2 + k_2k_3$ can recover k_1, k_2, k_3, k_4 easily and the number of chosen plaintexts is also reduced.

4.2 Non-linear Equations in Round 3 of PRESENT

Applying the scheme on PRESENT-80, we can extract additional non-linear superpolys and deduce more key bits. The results of SCCA with non-linear equations on L_3^{12} of PRESENT-80 are listed in Table 4. We can extract 16 extra non-linear equations to recover 16 more key bits. Since all the cube indexes in Table 4 are already in Table 3, Table 4 does not require new chosen plaintexts. Combined with the results in Sect. 3, 64 key bits of PRESENT-80 can be recovered.

Under SBLM in round 3, we have also conducted SCCA on PRESENT-128. At most the 64-bit first round key can be recovered.

In SCCA, the leakage bit in deeper rounds(e.g., in round 4) can involve more key bits and enable more efficient attacks. However, the polynomials in deeper rounds are too large and cannot be handled by the existing techniques in SCCA. In Sect. 5 and Sect. 6, we propose two techniques to handle large polynomials.

5. Enhanced SCCA with Partitioning

5.1 Locate the Leakage Bit in Round 4

We first need to select a leakage bit in round 4, where $D_{eg}(L_4^j)$ and $N_m(L_4^j)$ are smaller than the bits in later rounds. Figure 1 shows that $D_{eg}(L_4^0)$ is the lowest among all L_4^j s. As for $N_m(L_4^j)$, we can learn that that $N_m(L_4^{4d})$ is much smaller than $N_m(L_4^{4d+1}), N_m(L_4^{4d+2}), N_m(L_4^{4d+3})$ for $0 \leq d \leq 15$. Moreover, among all 16 $N_m(L_4^{4d}), 0 \leq d \leq 15$,

$N_m(L_4^0)$ is the smallest. Note that L_4^0 can be computed from $(L_3^0, L_3^4, L_3^8, L_3^{12})$, which have the smallest D_{eg} and N_m in round 3. So it is reasonable to choose L_4^0 as the leakage bit.

5.2 Partitioning Schemes

Standard SCCA cannot work with L_4^0 as both $D_{eg}(L_4^0)$ and $N_m(L_4^0)$ are large. We propose a partitioning scheme to cope with the large polynomials. The key point of the proposed scheme is how to cut the large polynomials into several pieces while the related cubes can still be extracted independently.

By observing the extracted maxterms and superpolys, we find out that both the number and the summations of the plaintext indexes are equal in the merged monomials. Based on this, the partitioning scheme is proposed in this paper. The partitioning scheme can be a single level based on $N_{S(I)}$ or two levels based on both $N_{S(I)}$ and the summations of indexes in I from Table 2. Which scheme to use depends on the performance requirement.

Single level partitioning scheme

For each cube size $N_{S(I)}$, a monomial in f is discarded if it has more than one key variable, or the number of the plaintext variables is larger than $N_{S(I)}$. Take Eq. (2) as an example. If $N_{S(I)} = 2$, applying the single level partitioning scheme, we can get $f(X) = v_1v_4(k_1 + k_3) + v_2v_4(k_4 + 1) + v_3v_4(k_2 + 1) + v_1k_1 + v_3 + v_4 + 1$.

Two-level partitioning scheme

After the single level partitioning, if the polynomial scale is still huge for representation, we can add a second level partitioning scheme. For the two-level partitioning scheme, the polynomial can be partitioned into several cases by three parameters, the cube size $N_{S(I)}$, the sum of plaintext indexes $S_{S(I)}$, and the empirical parameter L .

Let the plaintext index be in $[0, \lambda)$ where λ is the block size of the cipher. As to PRESENT, $\lambda = 64$. For each $N_{S(I)}$,

the minimal $S_{S(I)}$ is $\sum_{i=0}^{N_{S(I)}-1} i$ and the maximal $S_{S(I)}$ is $\sum_{i=\lambda-N_{S(I)}}^{\lambda-1} i$.

The empirical parameter L is introduced to cut the polynomial f into several smaller pieces and used to extract the maxterms and superpolys. Then the polynomial f can be

divided into $\varepsilon = \lceil (1 + \sum_{i=\lambda-N_{S(I)}}^{\lambda-1} i - \sum_{i=0}^{N_{S(I)}-1} i) / L \rceil$ parts. In the j -th

part ($1 \leq j \leq \varepsilon$), a monomial is kept if and only if it has at most one key variable, at most $N_{S(I)}$ plaintext variables, and the sum of plaintext indexes (denoted as $S_{S(I)}$) satisfies

$$S_{S(I)} \in \left[\sum_{i=0}^{N_{S(I)}-1} i + (j-1) \cdot L, \sum_{i=0}^{N_{S(I)}-1} i + j \cdot L \right) \quad (4)$$

Again let us take Eq. (2) as an example. Note that the plaintext index is started from 1, and the index is $[1, 4]$. If $L=2$, $j=4$, then the $S_{S(I)} \in [7, 8]$. We can get $f(X) = v_3v_4(k_2 + 1)$, both the degree and the number of monomials have been further reduced. It becomes easy to extract

Table 5 Cube indexes and superpolys on L_4^0 (PRESENT-128).

I	$PS(I)$	I	$PS(I)$
2, 3, 14, 15	$k_{64} + k_{76}$	0, 2, 3, 28, 29, 32, 33, 34	$k_{94} + k_{95}$
2, 3, 50, 51	$k_{64} + k_{112}$	0, 1, 2, 28, 29, 32, 33, 35	$1 + k_{95}$
2, 3, 62, 63	$k_{64} + k_{124}$	0, 1, 3, 16, 17, 18, 32, 34	$k_{97} + k_{99}$
1, 2, 12, 13, 15	k_{64}	0, 2, 3, 16, 17, 18, 32, 33	$k_{98} + k_{99}$
0, 3, 12, 13, 14	$1 + k_{65} + k_{66}$	0, 1, 2, 16, 17, 19, 32, 33	$1 + k_{99}$
0, 2, 12, 13, 14	k_{67}	0, 1, 3, 16, 17, 18, 44, 46	$k_{109} + k_{111}$
0, 1, 2, 12, 15	$1 + k_{77} + k_{78}$	0, 2, 3, 16, 17, 18, 44, 45	$k_{110} + k_{111}$
0, 1, 2, 12, 13	$1 + k_{79}$	0, 1, 2, 16, 17, 19, 44, 45	$1 + k_{111}$
0, 1, 2, 48, 51	$1 + k_{113} + k_{114}$	0, 1, 3, 52, 54, 56, 57, 58	$k_{117} + k_{119}$
0, 1, 2, 48, 49	$1 + k_{115}$	0, 2, 3, 52, 53, 56, 57, 58	$k_{118} + k_{119}$
0, 1, 2, 60, 63	$1 + k_{125} + k_{126}$	0, 1, 2, 52, 53, 56, 57, 59	$1 + k_{119}$
0, 1, 2, 60, 61	$1 + k_{127}$	0, 1, 3, 52, 53, 45, 56, 58	$k_{121} + k_{123}$
2, 3, 6, 7, 8, 9, 11	$k_{64} + k_{68}$	0, 2, 3, 52, 53, 45, 56, 57	$k_{122} + k_{123}$
0, 1, 3, 6, 7, 10, 11	$1 + k_{68} + k_{72}$	0, 1, 2, 52, 53, 55, 56, 57	$1 + k_{123}$
0, 1, 3, 18, 19, 34, 35	$1 + k_{80} + k_{96}$	22, 23, 24, 25, 27, 34, 35, 48, 49, 51	$1 + k_{84} + k_{96}$
0, 1, 3, 30, 31, 34, 35	$1 + k_{92} + k_{96}$	20, 22, 23, 26, 27, 34, 35, 48, 50, 51	$1 + k_{88} + k_{96}$
2, 3, 16, 17, 19, 34, 35	$k_{64} + k_{96}$	16, 17, 19, 38, 39, 40, 41, 43, 50, 51	$k_{100} + k_{112}$
0, 1, 3, 18, 19, 46, 47	$1 + k_{80} + k_{108}$	16, 17, 19, 36, 37, 39, 42, 43, 50, 51	$k_{104} + k_{112}$
0, 1, 3, 54, 55, 58, 59	$1 + k_{116} + k_{120}$	0, 1, 3, 20, 22, 24, 25, 26, 32, 33, 34	$k_{88} + k_{87}$
2, 3, 52, 53, 55, 58, 59	$k_{64} + k_{120}$	0, 1, 3, 20, 21, 24, 25, 27, 32, 33, 34	$k_{86} + k_{87}$
0, 1, 3, 4, 6, 8, 9, 10	$k_{69} + k_{71}$	0, 1, 2, 20, 21, 24, 25, 27, 32, 33, 35	$1 + k_{87}$
0, 1, 3, 4, 5, 8, 9, 10	$k_{70} + k_{71}$	0, 1, 3, 20, 21, 22, 24, 26, 32, 33, 34	$k_{89} + k_{91}$
0, 1, 2, 4, 5, 8, 9, 11	$1 + k_{71}$	0, 1, 3, 20, 21, 22, 25, 26, 32, 33, 35	$k_{90} + k_{91}$
0, 1, 3, 4, 5, 6, 8, 10	$k_{73} + k_{75}$	0, 1, 2, 20, 21, 23, 24, 25, 32, 34, 35	$1 + k_{91}$
0, 2, 3, 4, 5, 6, 8, 9	$k_{74} + k_{75}$	0, 1, 3, 16, 17, 18, 36, 38, 40, 41, 42	$k_{101} + k_{103}$
0, 1, 2, 4, 5, 7, 8, 9	$1 + k_{75}$	0, 1, 3, 16, 17, 18, 36, 37, 40, 41, 43	$k_{102} + k_{103}$
0, 1, 3, 16, 18, 32, 33, 34	$k_{81} + k_{83}$	0, 1, 2, 16, 17, 19, 36, 37, 40, 41, 43	$1 + k_{103}$
0, 2, 3, 16, 17, 32, 33, 34	$k_{82} + k_{83}$	0, 1, 3, 16, 17, 18, 36, 37, 38, 40, 42	$k_{105} + k_{107}$
0, 1, 2, 16, 17, 32, 33, 35	$1 + k_{83}$	0, 2, 3, 16, 17, 18, 36, 37, 38, 40, 41	$k_{106} + k_{107}$
0, 1, 3, 28, 30, 32, 33, 34	$k_{93} + k_{95}$	0, 1, 2, 16, 17, 19, 36, 37, 39, 40, 41	$1 + k_{107}$

the maxterm $t_I = v_3v_4$ and superpoly $p_{S(I)} = k_2 + 1$. More superpolys can be obtained with different $N_{S(I)}$ and L .

5.3 Application to PRESENT-128

In SCCA on PRESENT-128 based on SBLM in round 4, we adopt the two-level partitioning scheme. We set $L = 16$ and compute the 11 cases for $N_{S(I)} \in \{1, 2, \dots, 11\}$. The results of SCCA on L_4^0 are shown in Table 5.

When $N_{S(I)} = 4, 5, 7, 8, 10, 11$, $2^{14.56}$ chosen plaintexts can recover 56 key bits ($k_i, i \in \{64, 65, \dots, 127\} - A$, $A = \{65, 66, 77, 78, 113, 114, 125, 126\}$) and the XOR of four pairs of bits: $k_{65} + k_{66}, k_{77} + k_{78}, k_{113} + k_{114}, k_{125} + k_{126}$. The value of $k_i (i \in A)$ is still unknown. If $N_{S(I)} \geq 12$, it is still possible to recover all 64 bits of K^1 . Since N_{cp} increases greatly with $N_{S(I)}$, it is not desirable to choose a large $N_{S(I)}$. So we try to exploit more keys via another technique in Sect. 6.

6. Iterative SCCA

The main idea of iterative SCCA is to reconstruct the function of the targeted polynomial f by replacing the complicated algebraic representations of some intermediate bits which are composed by the plaintext bits and the recovered key bits with new equivalent plaintext bits.

As fewer unknown key variables are present in f , both D_{eg} and N_m are reduced. Then the complexity of f is reduced and the adversary can redo the SCCA on the reduced f . More maxterms and superpolys can then be deduced. Note that in iterative SCCA, the cubes are related with the new equivalent plaintext bits. The value of the chosen plaintexts may have minor changes from the plaintext bits in original SCCA and can be derived from the new equivalent

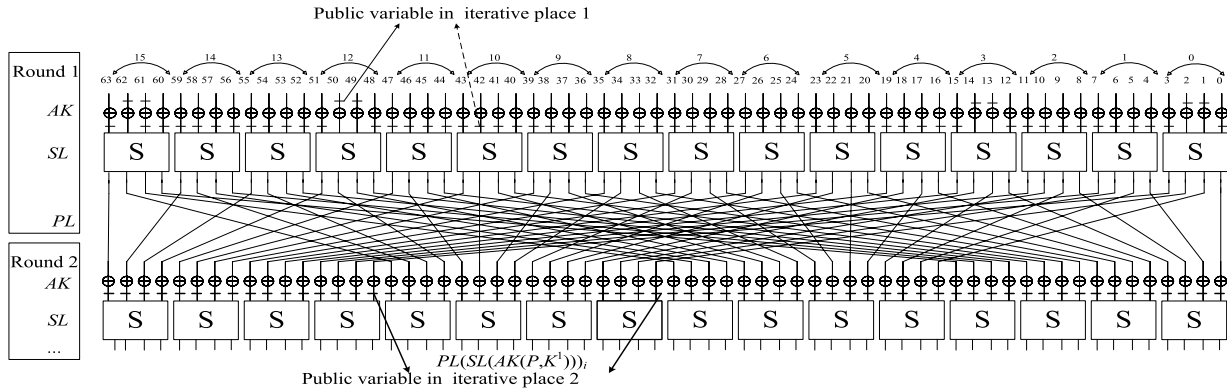


Fig. 3 Locations of the public variables in two iterative SCCAs.

Table 6 Cube indexes and superpolys on L_3^{12} (PRESENT-80, Iterative SCCA).

I	$PS(I)$	I	$PS(I)$
50	$1 + k_4$	52, 58	$1 + k_{12}$
49	k_5	52, 57	k_{13}
56	$1 + k_7 + k_9 + k_{10} + k_8 k_9 + k_{67}$	54, 56	$1 + k_8$
52	$k_{11} + k_{13} + k_{14} + k_{12} k_{13} + k_{68}$	52, 56	k_9

plaintext bits and some recovered key bits.

The proposed iterative SCCA is a generic technique to enhance previous SCCAs. Next, we apply it in attacks on both PRESENT-80 and PRESENT-128.

6.1 Iterative SCCA on PRESENT-80

Based on SBLM in round 3, we can recover 64-bit first round key K^1 of PRESENT-80 in Sect. 4. To obtain more key bits, we treat the output of the first round ($(PL(SL(AK(P, K^1))))$) as the new equivalent plaintext bits, as shown in the iterative place 2 of Fig. 3.

Then, the complicated representations of the 64 output bits of the first round can be replaced with 64 new equivalent plaintext bits. The original three rounds SCCA becomes a new two rounds SCCA on PRESENT-80. In the attack, the plaintexts are generated from the first round key K^1 and the new equivalent plaintext bits ($(PL(SL(AK(P, K^1))))$).

In iterative SCCA, the scale of f is reduced a lot. Thus more key bits can be recovered. Especially for L_3^{12} (several unrecovered key bits are involved in $f(L_3^{12})$), $2^{3,46}$ chosen plaintexts can recover eight more key bits, further reducing the PRESENT-80 key space to 2^8 , as shown in Table 6.

From Table 6, we can observe that some key bits which are recovered in previous SCCA may also exist in the new extracted superpolys (e.g., row 3 and row 4). This is because that some of the recovered key bits are also involved in the second and the third round key.

6.2 Iterative SCCA on PRESENT-128

In iterative SCCA on PRESENT. We consider SBLM in round 3 and SBLM in round 4 two different leakage models.

1. Iterative SCCA on PRESENT-128 based on

Table 7 Cube indexes and superpolys on L_3^4 (PRESENT-128).

I	K_{SI}	I	K_{SI}
1, 3	$1 + k_{64}$	16, 17, 18, 33, 35	$1 + k_{96}$
0, 2	$k_{65} + k_{67}$	16, 17, 18, 32, 34	$k_{97} + k_{99}$
0, 1	$k_{66} + k_{67}$	16, 17, 18, 32, 33	$k_{98} + k_{99}$
13, 15	$1 + k_{76}$	16, 17, 18, 45, 47	$1 + k_{108}$
12, 14	$k_{77} + k_{79}$	16, 17, 18, 44, 46	$k_{109} + k_{111}$
12, 13	$k_{78} + k_{79}$	16, 17, 18, 44, 45	$k_{110} + k_{111}$
49, 51	$1 + k_{112}$	53, 55, 56, 57, 58	$1 + k_{116}$
48, 50	$k_{113} + k_{115}$	52, 54, 56, 57, 58	$k_{117} + k_{119}$
48, 49	$k_{114} + k_{115}$	52, 53, 56, 57, 58	$k_{118} + k_{119}$
61, 63	$1 + k_{124}$	52, 53, 54, 57, 59	$1 + k_{120}$
60, 62	$k_{125} + k_{127}$	52, 53, 54, 56, 68	$k_{121} + k_{123}$
60, 61	$k_{126} + k_{127}$	52, 53, 54, 56, 67	$k_{122} + k_{123}$
5, 7, 8, 9, 10	$1 + k_{68}$	21, 23, 24, 25, 26, 32, 33, 34	$1 + k_{84}$
4, 6, 8, 9, 10	$k_{69} + k_{71}$	20, 22, 24, 25, 26, 32, 33, 34	$k_{85} + k_{87}$
4, 5, 8, 9, 10	$k_{70} + k_{71}$	20, 21, 24, 25, 26, 32, 33, 34	$k_{86} + k_{87}$
4, 5, 6, 9, 11	$1 + k_{72}$	20, 21, 22, 25, 27, 32, 33, 34	$1 + k_{88}$
4, 5, 6, 8, 10	$k_{73} + k_{75}$	20, 21, 22, 24, 26, 32, 33, 34	$k_{89} + k_{91}$
4, 5, 6, 8, 9	$k_{74} + k_{75}$	20, 21, 22, 24, 25, 32, 33, 34	$k_{90} + k_{91}$
17, 19, 32, 33, 34	$1 + k_{80}$	16, 17, 18, 37, 39, 40, 41, 42	$1 + k_{100}$
16, 18, 32, 33, 34	$k_{81} + k_{83}$	16, 17, 18, 36, 38, 40, 41, 42	$k_{101} + k_{103}$
16, 17, 32, 33, 34	$k_{82} + k_{83}$	16, 17, 18, 36, 37, 40, 41, 42	$k_{102} + k_{103}$
29, 31, 32, 33, 34	$1 + k_{92}$	16, 17, 18, 36, 37, 38, 41, 43	$1 + k_{104}$
28, 30, 32, 33, 34	$k_{93} + k_{95}$	16, 17, 18, 36, 37, 38, 40, 42	$k_{105} + k_{107}$
28, 29, 32, 33, 34	$k_{94} + k_{95}$	16, 17, 18, 36, 37, 38, 40, 41	$k_{106} + k_{107}$

SBLM in round 3

With the technique in Sect. 3 and Sect. 4, we can recover the 64-bit first round key K^1 with SCCA on L_3^4 of PRESENT-128. The results of SCCA by extracting linear and non-linear superpolys are shown in Table 7 and Table 8, respectively.

Then, we conduct the iterative SCCA on PRESENT-128 based on SBLM in round 3, 10 more bits can be recovered. The results are shown in Table 9.

2. Iterative SCCA on PRESENT-128 based on SBLM in round 4

Let us take SCCA on L_4^0 as an example. With the technique in Sect. 3 and Sect. 5, we can recover 56-bit k_i of PRESENT-128 as mentioned in Sect. 5. The results are shown in Table 5.

To obtain more key bits, we apply iterative SCCA on PRESENT-128. A notation P_i is introduced to represent the new equivalent public variables. In the first run of iterative

Table 8 Cube indexes and nonlinear superpolys on L_3^4 (PRESENT-128).

I	K_{SI}
1	$1 + k_{67} + k_{64}k_{66} + k_{64}k_{67}$
6, 8, 9, 10	$k_{71} + k_{68}k_{69} + k_{68}k_{71}$
4, 5, 6, 10	$k_{75} + k_{72}k_{73} + k_{72}k_{75}$
14	$k_{79} + k_{76}k_{77} + k_{76}k_{79}$
18, 32, 33, 34	$k_{83} + k_{80}k_{81} + k_{80}k_{83}$
22, 24, 25, 27, 32, 33, 34	$k_{87} + k_{84}k_{85} + k_{84}k_{87}$
20, 21, 22, 26, 32, 33, 34	$k_{91} + k_{88}k_{89} + k_{88}k_{91}$
30, 32, 33, 34	$k_{95} + k_{92}k_{93} + k_{92}k_{95}$
16, 17, 19, 34	$k_{99} + k_{96}k_{97} + k_{96}k_{99}$
16, 17, 19, 38, 40, 41, 42	$k_{103} + k_{100}k_{101} + k_{100}k_{103}$
16, 17, 18, 36, 37, 39, 42	$k_{107} + k_{104}k_{105} + k_{104}k_{107}$
16, 17, 18, 46	$k_{111} + k_{108}k_{109} + k_{108}k_{111}$
50	$k_{115} + k_{112}k_{113} + k_{112}k_{115}$
54, 56, 57, 58	$k_{119} + k_{116}k_{117} + k_{116}k_{119}$
52, 53, 54, 58	$k_{123} + k_{120}k_{121} + k_{120}k_{123}$
62	$k_{127} + k_{124}k_{125} + k_{124}k_{127}$

Table 9 Cube indexes and superpolys on L_3^4 (PRESENT-128, Iterative SCCA).

I	K_{SI}	I	K_{SI}
17	k_{21}	30	k_{32}
18	$1 + k_{20}$	20, 26	$1 + k_{28}$
20	$k_{27} + k_{29} + k_{30} + k_{28}k_{29} + k_{76}$	20, 25	k_{29}
24	$1 + k_{23} + k_{25} + k_{26} + k_{24}k_{25} + k_{75}$	20, 24	$1 + k_{24}$
29	k_{33}	21, 24	k_{25}

Table 10 Cube indexes and superpolys on L_4^0 (PRESENT-128, 1-st iterative SCCA).

I	$PS(I)$	I	$PS(I)$
0, 2, 12, 13, 15	k_{65}	0, 1, 3, 48, 50	k_{113}
0, 3, 12, 13, 14	$1 + k_{65} + k_{66}$	0, 1, 3, 48, 49	$1 + k_{114}$
0, 1, 3, 12, 13	$1 + k_{78}$	0, 1, 3, 60, 62	k_{125}
0, 1, 2, 12, 15	$1 + k_{77} + k_{78}$	0, 1, 3, 60, 61	$1 + k_{126}$

Table 11 Cube indexes and linear superpolys on L_4^0 (PRESENT-128, 2-nd iterative SCCA).

I	$PS(I)$	I	$PS(I)$	I	$PS(I)$	I	$PS(I)$
1	k_5	6, 8	$1 + k_8$	16, 33	k_{37}	22, 24, 32	$1 + k_{24}$
2	$1 + k_4$	4, 9	k_{13}	16, 34	$1 + k_{36}$	20, 25, 32	k_{29}
13	k_{17}	4, 10	$1 + k_{12}$	16, 45	k_{49}	20, 26, 32	$1 + k_{28}$
14	$1 + k_{16}$	17, 32	k_{21}	16, 46	$1 + k_{48}$	16, 37, 40	k_{41}
49	k_{53}	18, 32	$1 + k_{20}$	53, 56	k_{57}	16, 38, 40	$1 + k_{40}$
50	$1 + k_{52}$	29, 32	k_{33}	54, 56	$1 + k_{56}$	16, 36, 41	k_{45}
5, 8	k_9	30, 32	$1 + k_{32}$	21, 24, 32	k_{25}	16, 36, 42	$1 + k_{44}$

SCCA, the positions of P_i are shown in the iterative place 1 of Fig. 3. If $(i + 64) \in \{64, 65, \dots, 127\} - A$, P_i is updated with $v_i + k_{i+64}$, as shown in the dash arrow in Fig. 3; otherwise $P_i = v_i$, as shown in the solid arrow in Fig. 3.

After the first run of iterative SCCA, the 64-bit first round key of PRESENT-128 can be recovered. The results are shown in Table 10.

Then, we apply the second run of iterative SCCA on PRESENT-128 to recover more bits. The positions of P_i are shown in the iterative place 2 of Fig. 3. P_i has been updated with the output bit of the first round. Furthermore, by extracting linear superpolys, 28 more key bits can be obtained, as shown in Table 11.

Then, we consider to exploit the nonlinear superpolys

Table 12 Cube indexes and nonlinear superpolys on L_4^0 (PRESENT-128, 2-nd iterative SCCA).

I	$PS(I)$	I	$PS(I)$
8	$k_7 + k_9 + k_{10} + k_8k_9 + k_{71}$	16, 40	$1 + k_{39} + k_{41} + k_{42} + k_{40}k_{41} + k_{79}$
4	$k_{11} + k_{13} + k_{14} + k_{12}k_{13} + k_{72}$	16, 36	$k_{43} + k_{45} + k_{46} + k_{44}k_{45} + k_{80}$
24, 32	$1 + k_{23} + k_{25} + k_{26} + k_{24}k_{25} + k_{75}$	56	$1 + k_{55} + k_{57} + k_{58} + k_{56}k_{57} + k_{83}$
20, 32	$k_{27} + k_{29} + k_{30} + k_{28}k_{29} + k_{76}$		

in the second iterative SCCA, and 7 extra equivalent key bits can be retrieved, as shown in Table 12.

Overall, we can extract 99 key bits of PRESENT-128 with $2^{14.57}$ chosen plaintexts and the search space of PRESENT-128 is reduced to 2^{29} , in which a further brute-force attack can be launched with practical complexity.

7. Conclusions

In this paper, we discover that the method of counting chosen plaintexts in all previous works on SCCA is not accurate. Based on SBLM, we propose several techniques to enhance SCCA and apply it to attacks on PRESENT. To the best of our knowledge, these are the most efficient SCCAs on PRESENT-80/128. The key search space can be reduced to 2^8 for PRESENT-80 and to 2^{29} for PRESENT-128.

The cryptologists should carefully consider SCCA (cube attack) during the cipher design and evaluation phases. We notice that some recent lightweight ciphers are designed with SCCA resistance. Basically they try to mitigate SCCA from three aspects: (1) increase the algebraic degree for each output bit of the non-linear parts, e.g., Klein in [10]; (2) apply complicated linear parts, not just simple permutations, e.g., Klein in [10] and LED in [11]; (3) increase the rounds required to involve all the master key bits for the leakage bit, e.g., LBlock in [17].

References

- [1] S.F. Abdul-Latip, M.R. Reyhanitabar, W. Susilo, and J. Seberry, "On the security of NOEKEON against side channel cube attacks," Proc. 5th Information Security Practice and Experience Conference, Future Conference- ISPEC 2010, LNCS, vol.6047, pp.45–55, 2010.
- [2] S.F. Abdul-Latip, M.R. Reyhanitabar, W. Susilo, and J. Seberry, "Extended cubes: Enhancing the cube attack by extracting low-degree non-linear equations," Proc. 6th ACM Symposium on Information, Computer and Communications Security-ASIACCS 2011, pp.296–305, ACM society, 2011.
- [3] J.-P. Aumasson, I. Dinur, W. Meier, and A. Shamir, "Cube testers and key recovery attacks on reduced-round MD6 and trivium," Fast Software Encryption- FSE 2009, LNCS, vol.5665, pp.1–22, 2009.
- [4] G.V. Bard, N.T. Courtois, J. Nakahara, P. Sepehrdad, and B. Zhang, "Algebraic, AIDA/cube and side channel analysis of KATAN family of block ciphers," PROGRESS IN CRYPTOLOGY - INDOCRYPT 2010, LNCS, vol.6498, pp.176–196, 2010.
- [5] A. Bogdanov, L.R. Knudsen, G. Leander, C. Paar, A. Poschmann, M.J.B. Robshaw, Y. Seurin, and C. Vikkelsoe, PRESENT: An Ultra-Lightweight Block Cipher, in: Cryptographic Hardware and Embedded Systems- CHES 2007, LNCS, vol.4727, pp.450–466, 2007.
- [6] I. Dinur and A. Shamir, "Cube attacks on tweakable black-box polynomials," <http://eprint.iacr.org/2008/385>, 2008.
- [7] I. Dinur and A. Shamir, "Cube attacks on tweakable black-box polynomials," Advances in Cryptology-EUROCRYPT 2009, LNCS, vol.5479, pp.278–299, 2009.

- [8] I. Dinur and A. Shamir, "Side channel cube attacks on block ciphers," <http://eprint.iacr.org/2009/127>, 2009.
- [9] X. Fan and G. Gong, "On the security of hummingbird-2 against side channel cube attacks," *Proc. WEWoRC 2011*, pp.100–104, 2011.
- [10] Z. Gong, S.I. Nikova, and Y.W. Law, "KLEIN: A new family of lightweight block ciphers," Technical Report TR-CTIT-10-33, Centre for Telematics and Information Technology, University of Twente, Enschede, 2010.
- [11] J. Guo, T. Peyrin, A. Poschmann, and M. Robshaw, "The LED block cipher," *Cryptographic Hardware and Embedded Systems- CHES 2011, LNCS*, vol.6917, pp.326–341, 2011.
- [12] H. Handschuh, P. Paillier, and J. Stern, "Probing attacks on tamper-resistant devices," *Cryptographic Hardware and Embedded Systems- CHES 1999, LNCS*, vol.1717, pp.303–315, 1999.
- [13] P. Kocher, "Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems," *Proc. CRYPTO 1996, LNCS*, vol.1109, pp.104–113, 1996.
- [14] P. Kocher, J. Jaffe, and B. June, "Differential power analysis," *Proc. CRYPTO 1999, LNCS*, vol.1666, pp.388–397, 1999.
- [15] J.J. Quisquater and D. Samyde, "A new tool for non-intrusive analysis of smart cards based on electro-magnetic emissions: The SEMA and DEMA methods," *Eurocrypt Rump Session*, 2000.
- [16] M. Renaud, F. Standaert, and N. Veyrat-Charvillon, "Algebraic side-channel attacks on the AES: Why time also matters in DPA," *Proc. CHES 2009, LNCS*, vol.5747, pp.97–111, 2009.
- [17] W. Wu and L. Zhang, "LBlock: A lightweight block cipher," *Proc. 9th International Conference on Applied Cryptography and Network Security -ACNS 2011, LNCS*, vol.6715, pp.327–344, 2011.
- [18] L. Yang, M. Wang, and S. Qiao, "Side channel cube attack on PRESENT," *Proc. 8th International Conference on Cryptology And Network Security -CANS 2009, LNCS*, vol.5888, pp.379–391, 2009.
- [19] A. Zhang, C.-W. Lim, K. Khoo, W. Lei, and J. Pieprzyk, "Extensions of the cube attack based on low degree annihilators," <http://eprint.iacr.org/2009/049>, 2009.
- [20] X.J. Zhao, F. Zhang, S.Z. Guo, T. Wang, Z.J. Shi, H.Y. Liu, and K.K. Ji, "MDASCA: An Enhanced Algebraic Side-Channel Attack for Error Tolerance and New Leakage Model Exploitation," W. Schindler and S.A. Huss, eds., *COSADE 2012, LNCS 7275*, pp.231–248, 2012.



Xinjie Zhao received his B.E. degree and M.E. degree in Department of Computer Engineering, Ordnance Engineering College, Shijiazhuang, China, in 2006 and 2009, respectively. He is currently a Ph.D. student in Department of Computer Engineering, Ordnance Engineering College, Shijiazhuang, China. His main research interest includes side channel analysis, fault analysis and combined analysis of block ciphers.



Shize Guo was born in 1964. He received his Ph.D. degree in Harbin Institute of Technology, Harbin, China in 1989. He is currently a researcher in Institute of North Electronic Equipment. His main research interest includes information security and cryptography.



Fan Zhang was born in 1978. He received his Ph.D. degree in Department of Computer Science and Engineering from University of Connecticut in 2012. His research interests include side channel analysis and fault analysis in cryptography, computer architecture, security in wireless sensor network, etc.



Tao Wang was born in 1964. He received his Ph.D. degree in Institute of Computing Technology from Chinese Academy of Sciences in 1996. His research interests include information security and cryptography.



Zhijie Shi is an Assistant Professor of Computer Science and Engineering at the University of Connecticut (UConn). He received his Ph.D. degree from Princeton University in 2004 and his M.S. and B.S. degrees from Tsinghua University, China, in 1996 and 1992, respectively. Professor Shi received US National Science Foundation CAREER award in 2006. His research interests include hardware mechanisms for secure and reliable computing, side channel attacks and countermeasures, primitives for

ultra-efficient cipher designs, sensor network security, application specific instruction-set instruction processor (ASIP) and embedded system designs, and underwater sensor networks.



Hao Luo was born in 1980. He received his Ph.D. degree of instrument science and technology in Department of Electric Engineering and Automatics from Harbin Institute of Technology in 2008. He is currently working as a lecture in Zhejiang University. His research interests include image processing, information hiding and signal processing.